

Московский финансово-юридический университет МФЮА

Направление/специальность

Прикладная информатика в экономике

Отчет

По дисциплине:

Алгоритмизация и программирование

Автор:

Обучающийся 1 курса

Группы 28Прд4510

ФИО Остапенко Кирилл Николаевич

Руководитель:

Якушева Ирина Александровна

Отчет защищен с оценкой:

Сергиев Посад - 2026 г.

ЗАДАЧА № 4.32

1. Постановка задачи:

Дана квадратная матрица целых чисел $A(n, n)$. Требуется вычислить суммы элементов на каждой из диагоналей, параллельных главной диагонали (включая саму главную). Всего таких диагоналей $2n-1$. Результат вывести в порядке от самой верхней диагонали до самой нижней.

2. Описание алгоритма

Каждая диагональ, параллельная главной, определяется постоянной разностью индексов $d = i - j$.

Для главной диагонали $d = 0$

Для диагоналей выше $d = -1, -2, \dots, -(n-1)$

Для диагоналей ниже $d = 1, 2, \dots, n-1$

Чтобы обращаться к ним по порядку сверху вниз, введём индекс $k = i - j + (n - 1)$. Тогда k принимает значения от 0 до $2n-2$ ($k=0$ соответствует самой верхней диагонали, $k=n-1$ – главной, $k=2n-2$ – самой нижней).

Алгоритм:

Создаем массив `sums[2*MAXN-1]`, инициализируем нулями.

Проходим по всем i, j от 0 до $n-1$, вычисляем $k = i - j + n - 1$ и прибавляем $A[i][j]$ к `sums[k]`.

Выводим `sums[0] ... sums[2n-2]` через пробел.

3. Код Программы

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

const int MAX = 100;

int main() {
    setlocale(LC_ALL, "RUS");
    srand(time(0));

    int n;
    cout << "Введите размер матрицы n (не более " << MAX << "): ";
    cin >> n;

    int mode;
    cout << "Выберите режим ввода: 0 - с клавиатуры, 1 - случайная генерация: ";
    cin >> mode;

    int A[MAX][MAX];

    if (mode == 1) {
```

```

    cout << "Случайная матрица:\n";
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            A[i][j] = rand() % 21 - 10;
            cout << A[i][j] << " ";
        }
        cout << "\n";
    }
}
else {
    cout << "Введите элементы матрицы:\n";
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            cin >> A[i][j];
}

int sums[2 * MAX - 1] = { 0 };

for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j) {
        int k = i - j + n - 1;
        sums[k] += A[i][j];
    }

cout << "Суммы по диагоналям (сверху вниз):\n";
for (int k = 0; k < 2 * n - 1; ++k)
    cout << sums[k] << " ";
cout << endl;

return 0;
}

```

4. 3 примера для проверки работы:

```

Введите размер матрицы n (не более 100): 3
Выберите режим ввода: 0 – с клавиатуры, 1 – случайная генерация: 0
Введите элементы матрицы:
5
6
1
2
3
15
6
7
2
Суммы по диагоналям (сверху вниз):
1 21 10 9 6

```

```
Введите размер матрицы n (не более 100): 2
Выберите режим ввода: 0 - с клавиатуры, 1 - случайная генерация: 1
Случайная матрица:
0 -10
-1 -7
Суммы по диагоналям (сверху вниз):
-10 -7 -1

Введите размер матрицы n (не более 100): 5
Выберите режим ввода: 0 - с клавиатуры, 1 - случайная генерация: 1
Случайная матрица:
5 4 -2 3 2
-5 -9 -1 5 -2
9 -7 1 7 -8
7 -6 -8 -8 -8
4 7 10 -7 1
Суммы по диагоналям (сверху вниз):
2 1 -5 2 -10 -27 13 14 4
```

ЗАДАЧА № 4.14

1. Постановка задачи:

Дан одномерный массив T , состоящий из k целых чисел ($k \leq 100$). Требуется переставить его элементы в обратном порядке: первый элемент становится последним, второй – предпоследним и так далее. Результат вывести на экран. Программа также может генерировать массив случайным образом по выбору пользователя.

2. Описание алгоритма

Запрашиваем размер массива k .

Предлагаем пользователю выбор: ввод с клавиатуры (0) или случайная генерация (1).

При ручном вводе: в цикле вводим k чисел.

При случайной генерации: заполняем массив случайными числами от -10 до 10 , выводим исходный массив.

Затем переворачиваем массив: проходим по индексам от 0 до $k/2-1$, меняя местами выводим перевёрнутый массив. элементы i и $k-1-i$.

Выводим перевёрнутый массив.

3. Код Программы

```
#include <iostream>
#include <cstdlib>

#include <ctime>
using namespace std;

const int MAX = 100;

int main() {
    setlocale(LC_ALL, "RUS");
    srand(time(0));

    int k;
    cout << "Введите размер массива k (не более " << MAX << "): ";
    cin >> k;

    int mode;
    cout << "Режим ввода: 0 - с клавиатуры, 1 - случайная генерация: ";
    cin >> mode;

    int T[MAX];

    if (mode == 1) {
        cout << "Исходный массив: " << endl;
        for (int i = 0; i < k; ++i) {
            T[i] = rand() % 21 - 10;
            cout << T[i] << " ";
        }
        cout << " " << endl;
    }
    else {
```

```

    cout << "Введите элементы массива:" << endl;
    for (int i = 0; i < k; ++i)
        cin >> T[i];
}

for (int i = 0; i < k / 2; ++i) {
    int temp = T[i];
    T[i] = T[k - 1 - i];
    T[k - 1 - i] = temp;
}

cout << "Массив в обратном порядке: " << endl;
for (int i = 0; i < k; ++i)
    cout << T[i] << " ";
cout << endl;

return 0;
}

```

4. 3 примера для проверки работы

```

Введите размер массива k (не более 100): 5
Режим ввода: 0 – с клавиатуры, 1 – случайная генерация: 0
Введите элементы массива:
10 20 30 40 50
Массив в обратном порядке:
50 40 30 20 10
Введите размер массива k (не более 100): 4
Режим ввода: 0 – с клавиатуры, 1 – случайная генерация: 1
Исходный массив:
10 5 -1 -5
Массив в обратном порядке:
-5 -1 5 10

```

```

Введите размер массива k (не более 100): 1
Режим ввода: 0 – с клавиатуры, 1 – случайная генерация: 0
Введите элементы массива:
42
Массив в обратном порядке:
42

```

ЗАДАЧА № 4.34

1. Постановка задачи:

Для пригородного автобусного маршрута известно:

Время начала работы (часы и минуты)

Время окончания работы (часы и минуты)

Количество автобусов n

Протяжённость маршрута в одну сторону в минутах T_way

Время отдыха на конечных остановках в минутах T_rest

Требуется составить суточное расписание – моменты отправления с каждого из двух конечных пунктов (обозначим их А и В) в течение рабочего дня. Расписание не учитывает обед и пересменку. Автобусы работают равномерно с интервалом, зависящим от n и времени полного рейса.

2. Описание алгоритма

Перевести время начала и окончания в минуты от полуночи:

$start = \text{часы начала} * 60 + \text{минуты начала}$

$end = \text{часы окончания} * 60 + \text{минуты окончания}$

Вычислить время полного оборота (туда, отдых, обратно, отдых) для одного автобуса:

$full_cycle = 2 * (T_way + T_rest)$

Определить интервал между отправлениями с одного конечного пункта:

$interval = full_cycle / n$ (по условию – целое число минут)

Сформировать расписание отправлений из пункта А:

Первый рейс в $start$

Последующие – с шагом $interval$, пока время отправления меньше end (рейс считается, если автобус успевает отправиться до окончания работы)

Сформировать расписание отправлений из пункта В:

Первый рейс в $start + T_way + T_rest$ (первый автобус доезжает до В, отдыхает и отправляется обратно)

Далее с тем же шагом $interval$, пока время отправления меньше end

Вывести времена в формате ЧЧ: ММ (с ведущими нулями), отдельно для пункта А и пункта В.

3. Код Программы

```

#include <iostream>
using namespace std;

int main() {
    setlocale(LC_ALL, "RUS");
    int start_h, start_m, end_h, end_m;
    cout << "Введите время начала (часы минуты): ";
    cin >> start_h >> start_m;
    cout << "Введите время окончания (часы минуты): ";
    cin >> end_h >> end_m;

    int n, T_way, T_rest;
    cout << "Введите количество автобусов: ";
    cin >> n;
    cout << "Введите время в пути в одну сторону (мин): ";
    cin >> T_way;
    cout << "Введите время отдыха на конечной (мин): ";
    cin >> T_rest;

    int start = start_h * 60 + start_m;
    int end = end_h * 60 + end_m;

    int full_cycle = 2 * (T_way + T_rest);
    int interval = full_cycle / n;

    cout << "Отправления из пункта А:" << endl;
    for (int t = start; t < end; t += interval) {
        int h = t / 60;
        int m = t % 60;
        if (h < 10) cout << "0";
        cout << h << ":";
        if (m < 10) cout << "0";
        cout << m << " ";
    }
    cout << "Отправления из пункта В:" << endl;
    int first_B = start + T_way + T_rest;
    for (int t = first_B; t < end; t += interval) {
        int h = t / 60;
        int m = t % 60;
        if (h < 10) cout << "0";
        cout << h << ":";
        if (m < 10) cout << "0";
        cout << m << " ";
    }
    cout << endl;

    return 0;
}

```

4. 3 примера для проверки работы

```

Введите время начала (часы минуты): 6 0
Введите время окончания (часы минуты): 24 0
Введите количество автобусов: 2
Введите время в пути в одну сторону (мин): 30
Введите время отдыха на конечной (мин): 5

Отправления из пункта А:
06:00 06:35 07:10 07:45 08:20 08:55 09:30 10:05 10:40 11:15 11:50 12:25 13:00 13:35 14:10 14:45 15:20 15:55 16:30 17:05
17:40 18:15 18:50 19:25 20:00 20:35 21:10 21:45 22:20 22:55 23:30

Отправления из пункта В:
06:35 07:10 07:45 08:20 08:55 09:30 10:05 10:40 11:15 11:50 12:25 13:00 13:35 14:10 14:45 15:20 15:55 16:30 17:05 17:40
18:15 18:50 19:25 20:00 20:35 21:10 21:45 22:20 22:55 23:30

```


Введите время начала (часы минуты): 7 30
Введите время окончания (часы минуты): 23 0
Введите количество автобусов: 1
Введите время в пути в одну сторону (мин): 45
Введите время отдыха на конечной (мин): 10

Отправления из пункта А:
07:30 09:20 11:10 13:00 14:50 16:40 18:30 20:20 22:10

Отправления из пункта В:
08:25 10:15 12:05 13:55 15:45 17:35 19:25 21:15

Введите время начала (часы минуты): 5 0
Введите время окончания (часы минуты): 24 0
Введите количество автобусов: 3
Введите время в пути в одну сторону (мин): 15
Введите время отдыха на конечной (мин): 3

Отправления из пункта А:
05:00 05:12 05:24 05:36 05:48 06:00 06:12 06:24 06:36 06:48 07:00 07:12 07:24 07:36 07:48 08:00 08:12 08:24 08:36 08:48
09:00 09:12 09:24 09:36 09:48 10:00 10:12 10:24 10:36 10:48 11:00 11:12 11:24 11:36 11:48 12:00 12:12 12:24 12:36 12:48
13:00 13:12 13:24 13:36 13:48 14:00 14:12 14:24 14:36 14:48 15:00 15:12 15:24 15:36 15:48 16:00 16:12 16:24 16:36 16:48
17:00 17:12 17:24 17:36 17:48 18:00 18:12 18:24 18:36 18:48 19:00 19:12 19:24 19:36 19:48 20:00 20:12 20:24 20:36 20:48
21:00 21:12 21:24 21:36 21:48 22:00 22:12 22:24 22:36 22:48 23:00 23:12 23:24 23:36 23:48

Отправления из пункта В:
05:18 05:30 05:42 05:54 06:06 06:18 06:30 06:42 06:54 07:06 07:18 07:30 07:42 07:54 08:06 08:18 08:30 08:42 08:54 09:06
09:18 09:30 09:42 09:54 10:06 10:18 10:30 10:42 10:54 11:06 11:18 11:30 11:42 11:54 12:06 12:18 12:30 12:42 12:54 13:06
13:18 13:30 13:42 13:54 14:06 14:18 14:30 14:42 14:54 15:06 15:18 15:30 15:42 15:54 16:06 16:18 16:30 16:42 16:54 17:06
17:18 17:30 17:42 17:54 18:06 18:18 18:30 18:42 18:54 19:06 19:18 19:30 19:42 19:54 20:06 20:18 20:30 20:42 20:54 21:06
21:18 21:30 21:42 21:54 22:06 22:18 22:30 22:42 22:54 23:06 23:18 23:30 23:42 23:54

ЗАДАЧА № 4.20

1. Постановка задачи:

Дан многочлен степени n :

уравнение $P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$

Он задан массивом коэффициентов A размером $n+1$, где $A[0] = a_0, A[1] = a_1, \dots, A[n] = a_n$.

Требуется вычислить значение уравнение $P_n(x)$ для заданного вещественного x с использованием **схемы Горнера**, которая требует всего n операций умножения.

2. Описание алгоритма

Схема Горнера преобразует многочлен к виду:

$P_n(x) = (\dots((a_n x + a_{n-1})x + a_{n-2})x + \dots + a_1)x + a_0$

Алгоритм:

Инициализируем $result = A[n]$ (старший коэффициент).

Для $i = n-1$ до 0 (включительно):

$result = result * x + A[i]$

После цикла $result$ содержит значение многочлена.

Программа:

Ввод степени n (максимум 100).

Ввод коэффициентов от a_0 до a_n (всего $n+1$ число).

Ввод x .

Вычисление по схеме Горнера.

Вывод результата с точностью до 2 знаков (или просто как число).

Для удобства предусмотрен выбор режима ввода: 0 – ручной, 1 – случайная генерация коэффициентов (целые от -10 до 10) для демонстрации.

3. Код Программы

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <iomanip>
using namespace std;

const int MAX = 101;

int main() {
    setlocale(LC_ALL, "RUS");
    srand(time(0));
```

```

int n;
cout << "Введите степень многочлена n: ";
cin >> n;

int mode;
cout << "Режим ввода коэффициентов: 0 - ручной, 1 - случайный: ";
cin >> mode;

double A[MAX];

if (mode == 1) {
    cout << "Случайные коэффициенты (a0 ... a" << n << "):" << endl;
    for (int i = 0; i <= n; ++i) {
        A[i] = rand() % 21 - 10;
        cout << A[i] << " ";
    }
    cout << " " << endl;
}
else {
    cout << "Введите коэффициенты от a0 до a" << n << ": " << endl;
    for (int i = 0; i <= n; ++i)
        cin >> A[i];
}

double x;
cout << "Введите значение x: ";
cin >> x;

double result = A[n];
for (int i = n - 1; i >= 0; --i)
    result = result * x + A[i];

cout << fixed << setprecision(3);
cout << "Значение многочлена P(" << x << ") = " << result << endl;

return 0;
}

```

4. 3 примера для проверки работы

```

Введите степень многочлена n: 1
Режим ввода коэффициентов: 0 - ручной, 1 - случайный: 0
Введите коэффициенты от a0 до a1:
5 3
Введите значение x: 2
Значение многочлена P(2.000) = 11.000

```

Введите степень многочлена n: 2
Режим ввода коэффициентов: 0 – ручной, 1 – случайный: 0
Введите коэффициенты от a_0 до a_2 :
1 2 3
Введите значение x: 4
Значение многочлена $P(4.000) = 57.000$

Введите степень многочлена n: 3
Режим ввода коэффициентов: 0 – ручной, 1 – случайный: 1
Случайные коэффициенты ($a_0 \dots a_3$):
2 10 0 -3
Введите значение x: 1.5
Значение многочлена $P(1.500) = 6.875$

ЗАДАЧА № 5.10

1. Постановка задачи:

Вводится матрица целых чисел построчно с клавиатуры.

Количество строк заранее неизвестно и может быть велико.

Количество столбцов n фиксировано и вводится перед началом ввода строк.

Известно, что различных (уникальных) строк среди всех введённых не более m . Требуется: прочитать все строки матрицы. Сохранить в памяти только первые вхождения каждой уникальной строки (повторяющиеся строки игнорировать). Вывести полученную матрицу из уникальных строк в порядке их первого появления.

2. Описание алгоритма

Ввести количество столбцов n .

Ввести максимальное число уникальных строк m (гарантируется, что их не больше m).

Создать двумерный массив `unique` размером $m \times n$ для хранения уникальных строк, и счётчик `uniq_count` (сколько уже сохранено уникальных строк).

В цикле:

Ввести очередную строку (массив из n целых чисел).

Спросить пользователя, продолжать ли ввод (1 — да, 0 — нет). Если введён 0 — завершить ввод.

Проверить, совпадает ли введённая строка с какой-либо из уже сохранённых в `unique`. Для этого сравнить поэлементно.

Если такой строки ещё нет и `uniq_count < m`, то скопировать её в `unique[uniq_count]` и увеличить `uniq_count`.

Если строка уже есть — ничего не делать.

Если `uniq_count` достигло m , вывести предупреждение и завершить ввод.

После окончания ввода вывести все сохранённые уникальные строки.

3. Код Программы

```
#include <iostream>
using namespace std;
```

```

const int MAX_N = 100;
const int MAX_M = 100;

int main() {
    setlocale(LC_ALL, "RUS");

    int n, m;
    cout << "Введите количество столбцов: ";
    cin >> n;
    cout << "Введите максимальное количество различных строк: ";
    cin >> m;

    int unique[MAX_M][MAX_N];
    int uniq_count = 0;

    cout << "\nВвод строк матрицы (по n целых чисел в строке). " << endl;
    cout << "После ввода каждой строки введите 1 (продолжить) или 0 (закончить): " <<
endl;

    while (true) {
        int row[MAX_N];
        cout << "Введите строку: ";
        for (int j = 0; j < n; ++j)
            cin >> row[j];

        int cont;
        cout << "Продолжить? (1 - да, 0 - нет): ";
        cin >> cont;

        bool exists = false;
        for (int i = 0; i < uniq_count; ++i) {
            bool equal = true;
            for (int j = 0; j < n; ++j) {
                if (unique[i][j] != row[j]) {
                    equal = false;
                    break;
                }
            }
            if (equal) {
                exists = true;
                break;
            }
        }

        if (!exists) {
            if (uniq_count < m) {
                for (int j = 0; j < n; ++j)
                    unique[uniq_count][j] = row[j];
            }
        }
    }
}

```

```

        uniq_count++;
    }
    else {
        cout << "Достигнуто максимальное число уникальных строк! " << endl;
    }
}

if (cont == 0)
    break;
}

cout << "\nУникальные строки матрицы: " << endl;
for (int i = 0; i < uniq_count; ++i) {
    for (int j = 0; j < n; ++j)
        cout << unique[i][j] << " ";
    cout << endl;
}

return 0;
}

```

4. 3 примера для проверки работы

```

Введите количество столбцов: 2
Введите максимальное количество различных строк: 3

Ввод строк матрицы (по n целых чисел в строке).
После ввода каждой строки введите 1 (продолжить) или 0 (закончить):
Введите строку: 1 2
Продолжить? (1 - да, 0 - нет): 1
Введите строку: 3 4
Продолжить? (1 - да, 0 - нет): 1
Введите строку: 5 6
Продолжить? (1 - да, 0 - нет): 0

Уникальные строки матрицы:
1 2
3 4
5 6

```

```

Введите количество столбцов: 2
Введите максимальное количество различных строк: 3

Ввод строк матрицы (по n целых чисел в строке).
После ввода каждой строки введите 1 (продолжить) или 0 (закончить):
Введите строку: 1 1
Продолжить? (1 - да, 0 - нет): 1
Введите строку: 2 2
Продолжить? (1 - да, 0 - нет): 1
Введите строку: 1 1
Продолжить? (1 - да, 0 - нет): 1
Введите строку: 3 3
Продолжить? (1 - да, 0 - нет): 0

Уникальные строки матрицы:
1 1
2 2
3 3

```

```
Введите количество столбцов: 2
Введите максимальное количество различных строк: 2

Ввод строк матрицы (по n целых чисел в строке).
После ввода каждой строки введите 1 (продолжить) или 0 (закончить):
Введите строку: 1 2
Продолжить? (1 - да, 0 - нет): 1
Введите строку: 3 4
Продолжить? (1 - да, 0 - нет): 1
Введите строку: 5 6
Продолжить? (1 - да, 0 - нет): 1
Достигнуто максимальное число уникальных строк!
Введите строку: 1 2
Продолжить? (1 - да, 0 - нет): 0

Уникальные строки матрицы:
1 2
3 4
```


ЗАДАЧА № 5.23

1. Постановка задачи:

Дан массив A целых чисел, содержащий $2n$ элементов. Требуется получить два массива B и C , каждый размером n , следующим образом:

На каждом шаге из оставшихся элементов массива A выбрать два наиболее близких по значению (т. е. с минимальной абсолютной разностью)

Меньший из этих двух элементов поместить в массив B , больший — в массив C .

Выбранные элементы удаляются из дальнейшего рассмотрения.

Процесс повторяется, пока массивы B и C не будут полностью заполнены.

Если существует несколько пар с одинаковой минимальной разностью, можно выбрать любую (например, первую найденную).

2. Описание алгоритма

Ввести n (тогда размер A равен $2n$).

Ввести массив A из $2n$ целых чисел.

Создать массив `used` флагов (размер $2n$, все `false`), чтобы отметить уже использованные элементы.

Создать массивы B и C размером n .

Для $step = 0$ до $n-1$:

Установить `min_diff` в очень большое число (например, $2e9$).

Для всех пар (i, j) таких, что $i < j$, оба не использованы:

Вычислить разность $abs(A[i] - A[j])$.

Если разность меньше `min_diff`:

Обновить `min_diff`, запомнить i и j .

После поиска: пометить `used[i] = used[j] = true`.

Если $A[i] \leq A[j]$, то $B[step] = A[i]$, $C[step] = A[j]$, иначе $B[step] = A[j]$, $C[step] = A[i]$.

Вывести массивы B и C .

3. Код Программы

```
#include <iostream>
using namespace std;
```

```
const int MAX = 200;
```

```
int main() {
```

```

setlocale(LC_ALL, "RUS");

int n;
cout << "Введите n (размер B и C): ";
cin >> n;
int sizeA = 2 * n;

int A[MAX];
cout << "Введите " << sizeA << " элементов массива A: " << endl;
for (int i = 0; i < sizeA; ++i)
    cin >> A[i];

bool used[MAX] = { false };
int B[MAX / 2], C[MAX / 2];

for (int step = 0; step < n; ++step) {
    int min_diff = 2000000000;
    int idx1 = -1, idx2 = -1;

    for (int i = 0; i < sizeA; ++i) {
        if (used[i]) continue;
        for (int j = i + 1; j < sizeA; ++j) {
            if (used[j]) continue;
            int diff = (A[i] > A[j]) ? (A[i] - A[j]) : (A[j] - A[i]);
            if (diff < min_diff) {
                min_diff = diff;
                idx1 = i;
                idx2 = j;
            }
        }
    }

    used[idx1] = true;
    used[idx2] = true;

    if (A[idx1] <= A[idx2]) {
        B[step] = A[idx1];
        C[step] = A[idx2];
    }
    else {
        B[step] = A[idx2];
        C[step] = A[idx1];
    }
}

cout << "\nМассив B (меньшие элементы в парах): " << endl;
for (int i = 0; i < n; ++i)

```

```

        cout << B[i] << " ";
    cout << "\nМассив С (большие элементы в парах): " << endl;
    for (int i = 0; i < n; ++i)
        cout << C[i] << " ";
    cout << endl;

    return 0;
}

```

4. 3 примера для проверки работы

```

Введите n (размер В и С): 2
Введите 4 элементов массива А:
1 5 3 4

Массив В (меньшие элементы в парах):
4 1
Массив С (большие элементы в парах):
5 3

```

```

Введите n (размер В и С): 3
Введите 6 элементов массива А:
10 10 20 20 30 30

Массив В (меньшие элементы в парах):
10 20 30
Массив С (большие элементы в парах):
10 20 30

```

```

Введите n (размер В и С): 2
Введите 4 элементов массива А:
100 1 90 2

Массив В (меньшие элементы в парах):
1 90
Массив С (большие элементы в парах):
2 100

```

ЗАДАЧА № 5.32

1. Постановка задачи:

Дан трёхмерный массив $K[l][m][n]$, состоящий из нулей и единиц (1 — есть материал, 0 — пустота). Требуется получить три двумерные проекции (тени) на координатные плоскости:

XY (вид сверху) — размер $l \times m$: значение 1, если хотя бы по одному k элемент $K[i][j][k] = 1$.

XZ (вид спереди) — размер $l \times n$: значение 1, если хотя бы по одному j элемент $K[i][j][k] = 1$.

YZ (вид сбоку) — размер $m \times n$: значение 1, если хотя бы по одному i элемент $K[i][j][k] = 1$.

Ввод массива осуществляется с клавиатуры. Вывести три проекции построчно.

2. Описание алгоритма

Ввести размеры l, m, n .

Заполнить трёхмерный массив K вручную (нули и единицы).

Для проекции XY:

Создать массив $proj_xy[l][m]$, инициализировать нулями.

Для каждого i и j перебирать k , если найден $K[i][j][k] == 1$, установить $proj_xy[i][j] = 1$ и выйти из цикла по k .

Аналогично для XZ (перебор по j) и YZ (перебор по i).

Вывести все три проекции.

3. Код Программы

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

const int MAX = 100;

int K[MAX][MAX][MAX];

int main() {
    setlocale(LC_ALL, "RUS");
    srand(time(0));

    int l, m, n;
    cout << "Введите l, m, n (не более " << MAX << "): ";
    cin >> l >> m >> n;

    int mode;
```

```

cout << "Режим ввода (0 - ручной, 1 - случайный): ";
cin >> mode;

if (mode == 1) {
    for (int i = 0; i < l; ++i)
        for (int j = 0; j < m; ++j)
            for (int k = 0; k < n; ++k)
                K[i][j][k] = rand() % 2;
}
else {
    cout << "Введите " << l * m * n << " чисел (0 или 1):" << endl;
    for (int i = 0; i < l; ++i)
        for (int j = 0; j < m; ++j)
            for (int k = 0; k < n; ++k)
                cin >> K[i][j][k];
}

int xy[MAX][MAX] = { 0 };
for (int i = 0; i < l; ++i) {
    for (int j = 0; j < m; ++j) {
        for (int k = 0; k < n; ++k) {
            if (K[i][j][k] == 1) {
                xy[i][j] = 1;
                break;
            }
        }
    }
}

int xz[MAX][MAX] = { 0 };
for (int i = 0; i < l; ++i) {
    for (int k = 0; k < n; ++k) {
        for (int j = 0; j < m; ++j) {
            if (K[i][j][k] == 1) {
                xz[i][k] = 1;
                break;
            }
        }
    }
}

int yz[MAX][MAX] = { 0 };
for (int j = 0; j < m; ++j) {
    for (int k = 0; k < n; ++k) {
        for (int i = 0; i < l; ++i) {
            if (K[i][j][k] == 1) {
                yz[j][k] = 1;
                break;
            }
        }
    }
}

cout << "Проекция на XY (вид сверху):" << endl;
for (int i = 0; i < l; ++i) {
    for (int j = 0; j < m; ++j) {
        cout << xy[i][j] << " ";
    }
    cout << endl;
}

cout << "Проекция на XZ (вид спереди):" << endl;
for (int i = 0; i < l; ++i) {
    for (int k = 0; k < n; ++k) {

```

```

        cout << xz[i][k] << " ";
    }
    cout << endl;
}

cout << "Проекция на YZ (вид сбоку):" << endl;
for (int j = 0; j < m; ++j) {
    for (int k = 0; k < n; ++k) {
        cout << yz[j][k] << " ";
    }
    cout << endl;
}

return 0;
}

```

4. 3 примера для проверки работы

```

Введите l, m, n (не более 100): 3 3 3
Режим ввода (0 – ручной, 1 – случайный): 0
Введите 27 чисел (0 или 1):
0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0 0
Проекция на XY (вид сверху):
0 0 0
0 1 0
0 0 0
Проекция на XZ (вид спереди):
0 0 0
0 1 0
0 0 0
Проекция на YZ (вид сбоку):
0 0 0
0 1 0
0 0 0

```

```

Введите l, m, n (не более 100): 2 1 3
Режим ввода (0 – ручной, 1 – случайный): 0
Введите 6 чисел (0 или 1):
1 1 1
1 1 1
Проекция на XY (вид сверху):
1
1
Проекция на XZ (вид спереди):
1 1 1
1 1 1
Проекция на YZ (вид сбоку):
1 1 1

```

```
Введите l, m, n (не более 100): 2 2 2
Режим ввода (0 – ручной, 1 – случайный): 1
Проекция на XY (вид сверху):
1 1
1 1
Проекция на XZ (вид спереди):
1 1
1 1
Проекция на YZ (вид сбоку):
1 0
0 1
```

ЗАДАЧА № 6.17

1. Постановка задачи:

Дана квадратная матрица $A(n, n)$ целых чисел. В каждой строке матрицы найти наибольший элемент и поменять его местами с диагональным элементом этой строки (т.е. с $A[i][i]$). Если в строке несколько максимальных элементов, разрешается выбрать любой (например, первый встреченный). Вывести преобразованную матрицу.

2. Описание алгоритма

Ввести размер n (не более 100).

Ввести матрицу A построчно.

Для каждой строки i от 0 до $n-1$:

Найти индекс столбца j_max , в котором находится максимальный элемент в строке i .

Поменять местами $A[i][i]$ и $A[i][j_max]$.

Вывести полученную матрицу.

3. Код Программы

```
#include <iostream>
using namespace std;

const int MAX = 100;

int main() {
    setlocale(LC_ALL, "RUS");

    int n;
    cout << "Введите размер матрицы n (не более " << MAX << "): ";
    cin >> n;

    int A[MAX][MAX];

    cout << "Введите элементы матрицы:" << endl;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            cin >> A[i][j];

    for (int i = 0; i < n; ++i) {
        int j_max = 0;
        for (int j = 1; j < n; ++j) {
            if (A[i][j] > A[i][j_max])
                j_max = j;
        }
        int temp = A[i][i];
        A[i][i] = A[i][j_max];
        A[i][j_max] = temp;
    }

    cout << "Матрица после замен:" << endl;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j)
            cout << A[i][j] << " ";
    }
}
```



```
    cout << endl;  
}  
  
return 0;  
}
```

4. 3 примера для проверки работы

```
Введите размер матрицы n (не более 100): 2  
Введите элементы матрицы:  
3 1  
2 4  
Матрица после замен:  
3 1  
2 4
```

```
Введите размер матрицы n (не более 100): 3  
Введите элементы матрицы:  
1 2 3  
4 5 6  
7 8 9  
Матрица после замен:  
3 2 1  
4 6 5  
7 8 9
```

```
Введите размер матрицы n (не более 100): 3  
Введите элементы матрицы:  
5 5 1  
0 9 9  
2 2 2  
Матрица после замен:  
5 5 1  
0 9 9  
2 2 2
```

ЗАДАЧА № 6.15

1. Постановка задачи:

Дан массив M целых чисел размером k . В массиве много совпадающих элементов. Требуется найти количество различных элементов в этом массиве, не выполняя сортировку (т.е. не изменяя порядок элементов и не упорядочивая их). Вывести это количество.

2. Описание алгоритма

Ввести размер k (не более 100).

Ввести массив M из k целых чисел.

Создать счётчик $count = 0$.

Для каждого элемента $M[i]$ (i от 0 до $k-1$):

Проверить, встречался ли этот элемент ранее среди элементов $M[0] \dots M[i-1]$.

Если не встречался (т.е. элемент уникален среди предыдущих), увеличить $count$ на 1.

Вывести $count$

Этот алгоритм не требует дополнительной памяти под уникальные элементы (кроме переменных цикла) и не изменяет исходный массив.

3. Код Программы

```
#include <iostream>
using namespace std;

const int MAX = 100;

int main() {
    setlocale(LC_ALL, "RUS");

    int k;
    cout << "Введите размер массива k (не более " << MAX << "): ";
    cin >> k;

    int M[MAX];
    cout << "Введите элементы массива:" << endl;
    for (int i = 0; i < k; ++i)
        cin >> M[i];

    int count = 0;
    for (int i = 0; i < k; ++i) {
        bool found = false;
        for (int j = 0; j < i; ++j) {
            if (M[j] == M[i]) {
                found = true;
                break;
            }
        }
        if (!found)
            count++;
    }
}
```

```
cout << "Количество различных элементов: " << count << endl;  
  
return 0;  
}
```

4. 3 примера для проверки работы

```
Введите размер массива k (не более 100): 5  
Введите элементы массива:  
1 2 3 2 1  
Количество различных элементов: 3
```

```
Введите размер массива k (не более 100): 7  
Введите элементы массива:  
5 5 5 5 5 5 5  
Количество различных элементов: 1
```

```
Введите размер массива k (не более 100): 6  
Введите элементы массива:  
10 -5 0 10 0 3  
Количество различных элементов: 4
```

ЗАДАЧА № 6.8

1. Постановка задачи:

Дан массив $T(k)$ целых чисел. Найти первый и последний нулевые элементы в этом массиве (т.е. их порядковые номера/индексы). Если нулевых элементов нет, вывести соответствующее сообщение.

2. Описание алгоритма

Ввести размер k (не более 100).

Ввести массив T из k целых чисел.

Найти первый нулевой элемент:

Пройти по массиву от начала до конца, как только встретится $T[i] == 0$, запомнить индекс $first = i$ и выйти из цикла.

Найти последний нулевой элемент:

Пройти по массиву от конца к началу, как только встретится $T[i] == 0$, запомнить индекс $last = i$ и выйти из цикла.

Если $first$ не был найден (т.е. остался равным -1), вывести сообщение об отсутствии нулевых элементов.

Иначе вывести $first+1$ и $last+1$ (нумерация с 1) или сами индексы с 0 — на усмотрение. По условию «найти первый и последний нулевые элементы» — обычно выводят их порядковые номера (начиная с 1). Сделаем вывод с 1.

3. Код Программы

```
#include <iostream>
using namespace std;

const int MAX = 100;

int main() {
    setlocale(LC_ALL, "RUS");

    int k;
    cout << "Введите размер массива k (не более " << MAX << "): ";
    cin >> k;

    int T[MAX];
    cout << "Введите элементы массива:" << endl;
    for (int i = 0; i < k; ++i)
        cin >> T[i];

    int first = -1, last = -1;

    for (int i = 0; i < k; ++i) {
        if (T[i] == 0) {
            first = i;
            break;
        }
    }
```

```

    }

    for (int i = k - 1; i >= 0; --i) {
        if (T[i] == 0) {
            last = i;
            break;
        }
    }

    if (first == -1) {
        cout << "В массиве нет нулевых элементов" << endl;
    }
    else {
        cout << "Первый нулевой элемент: позиция " << first + 1 << endl;
        cout << "Последний нулевой элемент: позиция " << last + 1 << endl;
    }

    return 0;
}

```

4. 3 примера для проверки работы

```

Введите размер массива k (не более 100): 5
Введите элементы массива:
0 1 0 2 3
Первый нулевой элемент: позиция 1
Последний нулевой элемент: позиция 3

```

```

Введите размер массива k (не более 100): 4
Введите элементы массива:
1 2 3 4
В массиве нет нулевых элементов

```

```

Введите размер массива k (не более 100): 6
Введите элементы массива:
0 0 0 0 0 0
Первый нулевой элемент: позиция 1
Последний нулевой элемент: позиция 6

```

ЗАДАЧА № 6.30

1. Постановка задачи:

Даны два массива:

$K(n)$ – номера выигравших билетов (упорядочены по возрастанию);

$S(n)$ – суммы выигрышей для соответствующих номеров.

Также задана пачка купленных билетов с номерами $l_1, l_2, \dots, l_m$ (m – произвольное, номера могут быть в любом порядке). Требуется определить суммарный выигрыш по всем купленным билетам: если номер билета присутствует в массиве K , то добавляется соответствующая сумма из S , иначе – 0.

2. Описание алгоритма

Вводим количество выигравших билетов n , затем массивы K и S .

Вводим количество купленных билетов m , затем массив L из m номеров.

Суммарный выигрыш $total = 0$.

Для каждого номера $L[i]$:

Проходим по массиву K от 0 до $n-1$, ищем совпадение $K[j] == L[i]$.

Если нашли, добавляем $S[j]$ к $total$ и прекращаем поиск для этого номера.

Выводим $total$.

3. Код Программы

```
#include <iostream>
using namespace std;

const int MAX = 100;

int main() {
    setlocale(LC_ALL, "RUS");

    int n;
    cout << "Введите количество выигравших билетов n (не более " << MAX << "): ";
    cin >> n;

    int K[MAX], S[MAX];
    cout << "Введите номера выигравших билетов K:" << endl;
    for (int i = 0; i < n; ++i)
        cin >> K[i];
    cout << "Введите суммы выигрышей S:" << endl;
    for (int i = 0; i < n; ++i)
        cin >> S[i];
```

```

int m;
cout << "Введите количество купленных билетов m: ";
cin >> m;

int L[MAX];
cout << "Введите номера купленных билетов:" << endl;
for (int i = 0; i < m; ++i)
    cin >> L[i];

int total = 0;
for (int i = 0; i < m; ++i) {
    for (int j = 0; j < n; ++j) {
        if (K[j] == L[i]) {
            total += S[j];
            break;
        }
    }
}

cout << "Суммарный выигрыш: " << total << endl;

return 0;
}

```

4. 3 примера для проверки работы

```

Введите количество выигравших билетов n (не более 100): 3
Введите номера выигравших билетов K:
10 20 30
Введите суммы выигрышей S:
100 200 300
Введите количество купленных билетов m: 2
Введите номера купленных билетов:
20 40
Суммарный выигрыш: 200

```

```

Введите количество выигравших билетов n (не более 100): 4
Введите номера выигравших билетов K:
1 5 10 15
Введите суммы выигрышей S:
50 150 250 350
Введите количество купленных билетов m: 3
Введите номера купленных билетов:
10 1 20
Суммарный выигрыш: 300

```

```

Введите количество выигравших билетов n (не более 100): 2
Введите номера выигравших билетов K:
100 200
Введите суммы выигрышей S:
1000 2000
Введите количество купленных билетов m: 0
Введите номера купленных билетов:
Суммарный выигрыш: 0

```


ЗАДАЧА № 6.35

1. Постановка задачи:

Дана матрица $A(m, n)$ из нулей и единиц. Требуется решить две подзадачи:

Квадрат заданного размера (10 б.) – найти квадратную подматрицу размера $k \times k$, состоящую целиком из нулей. Если такая существует, вывести координаты её верхнего левого угла, иначе – сообщение об отсутствии.

Квадрат наибольшего размера (16 б.) – найти квадратную подматрицу максимального размера, состоящую целиком из нулей, вывести её размер и координаты верхнего левого угла.

Программа последовательно решает обе задачи для одной и той же матрицы.

2. Описание алгоритма

Общая часть:

Ввод размеров m, n (не более 100) и матрицы A из нулей и единиц.

Для квадрата заданного размера k :

Если $k > m$ или $k > n$, сразу сообщить, что не помещается.

Перебираем все возможные верхние левые углы (i, j) от 0 до $m-k$ и 0 до $n-k$.

Для каждого угла проверяем все $k \times k$ элементов. При первом найденном квадрате выводим координаты (с нумерацией с 1) и прекращаем поиск.

Если не нашли – выводим сообщение.

Для квадрата наибольшего размера:

Используем метод динамического программирования:

Создаём массив $dp[m][n]$, где $dp[i][j]$ – максимальный размер квадрата из нулей, у которого правый нижний угол находится в (i, j) .

Если $A[i][j] == 1$, то $dp[i][j] = 0$.

Иначе:

Если $i == 0$ или $j == 0$, то $dp[i][j] = 1$.

Иначе $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$.

В процессе заполнения запоминаем максимальное значение max_size и координаты (pos_i, pos_j) правого нижнего угла.

После заполнения выводим max_size и координаты верхнего левого угла:

$(pos_i - max_size + 2, pos_j - max_size + 2)$ (переход к нумерации с 1).

3. Код Программы

```
#include <iostream>
using namespace std;

const int MAX = 100;

int main() {
    setlocale(LC_ALL, "RUS");

    int m, n;
    cout << "Введите количество строк m и столбцов n (не более " << MAX << "): ";
    cin >> m >> n;

    int A[MAX][MAX];
    cout << "Введите матрицу из 0 и 1:" << endl;
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            cin >> A[i][j];

    int k;
    cout << "Введите размер искомого квадрата k: ";
    cin >> k;

    if (k > m || k > n) {
        cout << "Квадрат размера " << k << " не помещается в матрицу" << endl;
    }
    else {
        bool found = false;
        int row = -1, col = -1;
        for (int i = 0; i <= m - k; ++i) {
            for (int j = 0; j <= n - k; ++j) {
                bool all_zero = true;
                for (int x = i; x < i + k; ++x) {
                    for (int y = j; y < j + k; ++y) {
                        if (A[x][y] != 0) {
                            all_zero = false;
                            break;
                        }
                    }
                }
                if (!all_zero) break;
            }
            if (all_zero) {
                found = true;
                row = i;
                col = j;
            }
        }
    }
}
```

```

        break;
    }
}
if (found) break;
}
if (found) {
    cout << "Квадрат размера " << k << " из нулей найден" << endl;
    cout << "Верхний левый угол: строка " << row + 1 << ", столбец " << col + 1 << endl;
}
else {
    cout << "Квадрат размера " << k << " из нулей не найден" << endl;
}
}
}

int dp[MAX][MAX] = { 0 };
int max_size = 0;
int pos_i = -1, pos_j = -1;
for (int i = 0; i < m; ++i) {
    for (int j = 0; j < n; ++j) {
        if (A[i][j] == 1) {
            dp[i][j] = 0;
        }
        else {
            if (i == 0 || j == 0) {
                dp[i][j] = 1;
            }
            else {
                int t = dp[i - 1][j];
                if (dp[i][j - 1] < t) t = dp[i][j - 1];
                if (dp[i - 1][j - 1] < t) t = dp[i - 1][j - 1];
                dp[i][j] = t + 1;
            }
        }
        if (dp[i][j] > max_size) {
            max_size = dp[i][j];
            pos_i = i;
            pos_j = j;
        }
    }
}
if (max_size == 0) {
    cout << "В матрице нет нулей" << endl;
}
else {
    cout << "Наибольший квадрат из нулей имеет размер " << max_size << "x" << max_size << endl;
    cout << "Верхний левый угол: строка " << pos_i - max_size + 2 << ", столбец " << pos_j - max_size + 2 << endl;
}

return 0;
}

```

4. 3 примера для проверки работы

```
Введите количество строк m и столбцов n (не более 100): 4 4
Введите матрицу из 0 и 1:
0 0 1 0
0 0 0 1
1 0 0 0
0 1 0 0
Введите размер искомого квадрата k: 2
Квадрат размера 2 из нулей найден
Верхний левый угол: строка 1, столбец 1
Наибольший квадрат из нулей имеет размер 2x2
Верхний левый угол: строка 1, столбец 1
```

```
Введите количество строк m и столбцов n (не более 100): 5 5
Введите матрицу из 0 и 1:
1 1 1 1 1
1 0 0 0 1
1 0 0 0 1
1 0 0 0 1
1 1 1 1 1
Введите размер искомого квадрата k: 3
Квадрат размера 3 из нулей найден
Верхний левый угол: строка 2, столбец 2
Наибольший квадрат из нулей имеет размер 3x3
Верхний левый угол: строка 2, столбец 2
```

```
Введите количество строк m и столбцов n (не более 100): 3 3
Введите матрицу из 0 и 1:
1 1 1
1 1 1
1 1 1
Введите размер искомого квадрата k: 2
Квадрат размера 2 из нулей не найден
В матрице нет нулей
```

ЗАДАЧА № 4.8

1. Постановка задачи:

Дан массив $C(m)$ вещественных чисел (целые числа также допустимы). Требуется выполнить преобразование массива без использования дополнительного рабочего массива:

Каждый третий элемент (начиная с третьего, т.е. элементы с индексами 3, 6, 9, ... в нумерации с 1) заменить на полусумму двух предыдущих элементов (стоящих перед ним).

Элемент, стоящий непосредственно перед каждым таким третьим элементом (т.е. второй элемент в каждой тройке), заменить на полусумму своих соседей (первого и третьего элементов тройки).

Преобразования выполняются последовательно для каждой тройки, при этом для вычислений используются исходные значения элементов тройки (до изменений). Последние элементы, не образующие полную тройку, остаются без изменений.

2. Описание алгоритма

Ввести размер массива m .

Ввести элементы массива C (вещественные числа).

Пройти по массиву с шагом 3, начиная с индекса 2 (при 0-индексации соответствует третьему элементу):

Для каждого $i = 2, 5, 8, \dots$ (пока $i < m$):

Сохранить значения $a = C[i-2]$, $b = C[i-1]$, $c = C[i]$.

Вычислить новое значение для элемента $C[i-1]$ как $(a + c) / 2.0$.

Вычислить новое значение для элемента $C[i]$ как $(a + b) / 2.0$.

(Порядок присваивания не важен, так как используется сохранённое a , b , c).

Вывести преобразованный массив.

3. Код Программы

```
#include <iostream>
using namespace std;

const int MAX = 100;

int main() {
    setlocale(LC_ALL, "RUS");

    int m;
    cout << "Введите размер массива m (не более " << MAX << "): ";
    cin >> m;

    double C[MAX];
    cout << "Введите элементы массива:" << endl;
    for (int i = 0; i < m; ++i)
        cin >> C[i];

    for (int i = 2; i < m; i += 3) {
        double a = C[i - 2];
        double b = C[i - 1];
        double c = C[i];
        C[i - 1] = (a + c) / 2.0;
        C[i] = (a + b) / 2.0;
    }

    cout << "Преобразованный массив:" << endl;
    for (int i = 0; i < m; ++i)
        cout << C[i] << " ";
    cout << endl;

    return 0;
}
```

4. 3 примера для проверки работы

```
Введите размер массива m (не более 100): 6
Введите элементы массива:
1 2 3 4 5 6
Преобразованный массив:
1 2 1.5 4 5 4.5
```

```
Введите размер массива m (не более 100): 4
Введите элементы массива:
10 20 30 40
Преобразованный массив:
10 20 15 40
```

```
Введите размер массива m (не более 100): 3
Введите элементы массива:
0 0 0
Преобразованный массив:
0 0 0
```